

Pearls AQI Predictor

*A Serverless End-to-End Machine Learning System for
Three-Day Air Quality Index Forecasting*

Ameer Hamza

Technical Report

Study area	Karachi, Pakistan (24.8607° N, 67.0011° E)
Forecast horizons	+24 h, +48 h, +72 h
Feature store	8,835 hourly observations
Coverage	3 Sep 2025 – 6 Sep 2026 (368 days)
Deployed model	Random Forest (all three horizons)
Repository	https://github.com/AmeerHvmza/10pearls_AQI-predictor

6 September 2026

Abstract

This report documents the design, implementation and evaluation of the *Pearls AQI Predictor*, a fully operational machine learning system that forecasts the United States EPA Air Quality Index for Karachi, Pakistan at horizons of 24, 48 and 72 hours. The system is deliberately built to be *100% serverless*: every scheduled component executes on GitHub Actions runners, model artefacts and the feature store are versioned in the Git repository itself, and the user-facing dashboard is hosted on Streamlit Community Cloud with continuous deployment from the `main` branch. Air pollution measurements are obtained from the OpenWeather Air Pollution API and meteorological variables from the Open-Meteo historical archive; an hourly pipeline appends fresh observations to a dual-write feature store consisting of a local Apache Parquet table (the authoritative, dependency-free source of truth) and a managed Hopsworks feature group populated by stream ingestion. At the time of writing the store holds 8,835 consecutive hourly rows spanning 368 days, with no missing measured hours. A nightly training pipeline engineers 64 features, trains four candidate model families – Ridge regression, Random Forest, Gradient Boosting and a Keras LSTM – under five-fold expanding-window time-series cross-validation, and automatically deploys the best model per horizon only if it beats a persistence baseline. Random Forest won every horizon, reducing baseline RMSE by 7.0%, 10.5% and 8.1% at +24h, +48h and +72h respectively (RMSE 27.43, 34.98, 39.14 AQI points). Forecasts are explained with SHAP, served with uncertainty bands through a shared prediction module, and surfaced through both a FastAPI REST interface and a two-page Streamlit dashboard with EPA-threshold hazard alerting.

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Objectives	1
1.3	The 100% Serverless Architecture Goal	1
2	System Architecture	2
2.1	Layer Responsibilities	2
3	Data Pipeline	3
3.1	How Data Is Collected	3
3.2	Final Dataset	3
3.3	Feature Engineering	4
3.4	Exploratory Data Analysis	5
4	Feature Store	6
4.1	The Dual-Write Design	6
4.2	Tuning the Ingestion Method for the Deployment Environment	7
5	Model Training and Evaluation	7
5.1	Training Pipeline	7
5.2	Candidate Models	7
5.3	Predicting the Change Rather than the Level	8
5.4	Evaluation Protocol	8
5.5	Results	8
6	Explainability with SHAP	10
6.1	Why Explainability Matters Here	10
6.2	Methodology	10

7	Prediction Mechanism	13
8	Automation and CI/CD	15
8.1	Workflow Architecture	15
8.2	Evidence of Independent Scheduled Execution	15
9	Web Application	16
9.1	Streamlit Dashboard	16
9.2	FastAPI Backend	16
9.3	Deployed Interface	17
10	Alerts System	20
10.1	Threshold Logic	20
10.2	Surfacing the Alert	21
11	Deployment	21
12	Results Summary	22
13	Limitations and Future Work	23
14	Conclusion	24
	References	25

List of Figures

1	System architecture	2
2	Exploratory data analysis	6
3	SHAP summary, 24-hour horizon	11
4	SHAP summary, 48-hour horizon	12
5	SHAP summary, 72-hour horizon	13
6	Dashboard page	18
7	Model Analysis page	19
8	Alert banner, triggered state	20

List of Tables

1	Feature store characteristics (<code>data/features.parquet</code>).	4
2	Engineered feature groups.	5
3	Model comparison across horizons	9
4	Deployed model versus persistence baseline.	9
5	Scheduled workflows.	15
6	REST API endpoints.	16
7	EPA AQI categories as implemented, with alerting status.	20
8	Requirement fulfilment matrix.	22

1 Introduction

1.1 Problem Statement

Ambient air pollution is among the largest environmental risk factors for human health world-wide. Fine particulate matter (PM_{2.5}) penetrates deep into the respiratory tract and is causally associated with cardiovascular disease, respiratory illness, reduced lung function in children, and premature mortality. South Asian megacities are disproportionately affected, and Karachi – the study area for this project – routinely experiences Air Quality Index (AQI) values well into the ranges the US Environmental Protection Agency classifies as *Unhealthy for Sensitive Groups* or worse.

Crucially, the health burden of poor air quality is partially *avoidable*. Sensitive individuals – asthmatics, the elderly, young children, outdoor workers – can materially reduce their exposure if they know in advance that a pollution episode is coming: rescheduling outdoor exercise, using filtration indoors, or wearing appropriate masks. This makes AQI *forecasting*, as distinct from AQI *monitoring*, a directly actionable public-health capability. A dashboard that tells a user the air is already hazardous is informative; one that tells them the air will be hazardous in two days is useful.

The measured distribution for Karachi over the study period underlines the point. Across 8,835 hourly observations the mean AQI is 87.5, but the series ranges from a minimum of 14.8 to a maximum saturating the 500-point EPA scale – an order-of-magnitude swing that an occupant of the city cannot anticipate without a model.

1.2 Objectives

The project was scoped around seven concrete objectives:

1. Build an automated **feature pipeline** that ingests real air pollution and weather observations on an hourly cadence.
2. **Backfill** a substantial history of genuine measurements so that models can learn seasonal and diurnal structure.
3. Maintain a **feature store** that is simultaneously simple enough to run with zero external accounts and production-grade enough to demonstrate managed-feature-store practice.
4. Implement a **training pipeline** that evaluates multiple model families per horizon under honest time-series cross-validation and automatically promotes the winner to a model registry.
5. Make every forecast **explainable** using SHAP.
6. **Serve** forecasts through both a machine interface (REST API) and a human interface (interactive dashboard), including hazard alerting.
7. Run the whole system on a **100% serverless** footing, with no manual intervention.

1.3 The 100% Serverless Architecture Goal

A conventional deployment of this system would require at least one always-on virtual machine to host a scheduler, a database for the feature store, an object store for model artefacts, and a web server for the front end. That is four billable components and four things to keep patched.

The architectural thesis of this project is that a system of this class needs none of them. Instead:

- **Scheduling** is delegated to GitHub Actions **cron** triggers – an hourly feature job and a daily training job.

- **The feature store** is an Apache Parquet file committed to the repository, giving free versioning, free durability and free point-in-time recovery through Git history. A managed Hopsworks feature group is written in parallel for production parity.
- **The model registry** is the `models/` directory in the same repository: serialised estimators alongside machine-readable metrics and explainability artefacts, all version-controlled.
- **Serving** is a Streamlit Community Cloud application bound directly to the repository, rebuilt automatically on every push to `main`.

The result is a continuously-updating production ML system with an infrastructure cost of zero and no server for an operator to log into. The remainder of this report explains how each layer works and what the system achieves.

2 System Architecture

The system is organised as a strictly one-directional pipeline: external data sources feed a feature pipeline, which writes to a feature store, which is read by both a training pipeline and a serving layer. The training pipeline writes to a model registry, which the serving layer also reads. Automation drives the two pipelines on independent schedules. Figure 1 shows the complete data flow.

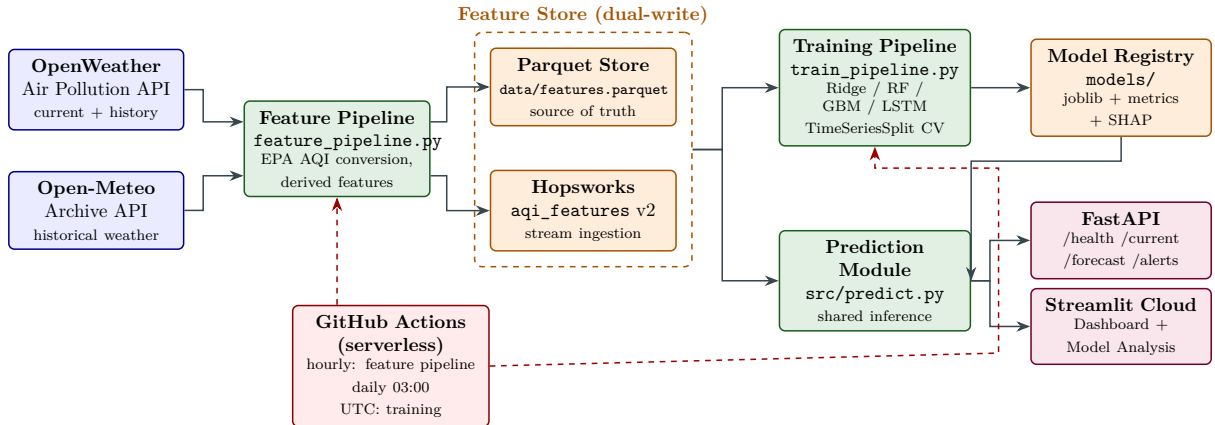


Figure 1: End-to-end architecture of the Pearls AQI Predictor. Solid arrows are data flow; dashed red arrows are scheduled invocation by GitHub Actions. The feature store is written twice (local Parquet plus a managed Hopsworks feature group) but read once, from Parquet. Both serving surfaces share a single prediction module, so the REST API and the dashboard cannot disagree.

2.1 Layer Responsibilities

Ingestion layer.

Two external providers are used for complementary reasons. OpenWeather supplies pollutant concentrations both in real time and as a queryable history back to late 2020, under a single free-tier key. Open-Meteo supplies the historical meteorological archive without requiring a key at all. The feature pipeline converts raw pollutant concentrations into a US EPA AQI value before anything downstream sees them.

Feature store layer.

A single Parquet table on a continuous hourly grid, keyed by UTC `timestamp`. Every writer goes through `utils.save_feature_store`, which merges, de-duplicates on timestamp, and then attempts the Hopsworks dual-write.

Training layer.

Reads the whole store, engineers features, runs cross-validated model selection per horizon, and writes exactly three artefact sets to the registry – one per horizon.

Registry layer.

Plain files: `model_{h}h.joblib` bundles (estimator, fitted scaler, feature column list, residual statistics, baseline verdict), `metrics_{h}h.json`, and the SHAP outputs. Because these are committed to Git, every retrain is an auditable diff.

Serving layer.

`src/predict.py` is the only code path that turns a feature store row into a forecast. FastAPI and Streamlit are both thin presentation wrappers over it.

Automation layer.

Two GitHub Actions workflows sharing a `concurrency` group, so the hourly and daily jobs can never collide when they both attempt to commit at 03:00 UTC.

3 Data Pipeline

3.1 How Data Is Collected

Data acquisition is split into a one-off historical backfill and a recurring hourly increment.

Historical backfill (`src/backfill.py`). Pollutant history is requested from the OpenWeather air-pollution history endpoint, `api.openweathermap.org/data/2.5/air_pollution/history`, in 30-day chunks to stay inside the endpoint’s per-request window. Weather history for the same interval is requested from the Open-Meteo archive, `archive-api.open-meteo.com/v1/archive`, which needs no API key. The two responses are joined on the UTC hour. Every backfill run is recorded in `data/PROVENANCE.md` together with the exact command, the number of successful HTTP 200 responses, the number of records returned by each provider, the number of rows actually written, and a SHA-256 checksum of the resulting Parquet file. The first raw JSON response from each provider is retained verbatim in `data/provenance_first_pollution.json` and `data/provenance_first_weather.json` as physical evidence that the history is measured rather than synthesised.

Hourly increment (`src/feature_pipeline.py`). Once per hour a GitHub Actions runner executes the feature pipeline, which fetches the current pollutant concentrations and current weather for the configured coordinates, converts them to an AQI, assembles a single feature row, and merges it into the store. The merge is idempotent: rows are de-duplicated on `timestamp`, so a re-run of the same hour updates rather than duplicates.

AQI computation. The AQI is not taken from the provider’s own 1–5 index. Instead `owm_aqi_to_us_aqi()` implements the EPA definition directly: the AQI is the *maximum* of the per-pollutant sub-indices computed over PM_{2.5}, PM₁₀, O₃, CO, SO₂ and NO₂. Gaseous species are converted from $\mu\text{g m}^{-3}$ to ppb or ppm at 25 °C and 1 atm before breakpoint lookup, and each concentration is truncated to its breakpoint table’s published precision – without truncation a PM_{2.5} reading of $12.03 \mu\text{g m}^{-3}$ falls in the gap between the 12.0 and 12.1 breakpoints and matches no bucket. Because concentrations are stored raw, the AQI column can be recomputed at any time by `src/repair_feature_store.py` if the conversion is refined.

3.2 Final Dataset

Table 1 summarises the feature store as it stands.

Table 1: Feature store characteristics (`data/features.parquet`).

Property	Value
Location	Karachi (24.8607° N, 67.0011° E)
Hourly rows	8,835
Rows with a measured AQI	8,835 (100%)
Imputed rows	0
Stored columns	27
Start of coverage (UTC)	2025-09-03 00:00
End of coverage (UTC)	2026-09-06 02:00
Span	368 days
Maximum inter-observation gap	1 hour
Mean AQI	87.52
Minimum / maximum AQI	14.8 / 500.0
Mean PM _{2.5}	31.90 $\mu\text{g m}^{-3}$
File size	504,596 byte

The store was assembled incrementally, and `data/PROVENANCE.md` records each stage: an initial 90-day backfill yielding 2,055 rows (2026-06-03 to 2026-08-27), a subsequent 365-day pull merged to 8,761 rows, and a consolidation and AQI-recomputation pass producing the unified 8,825-row store with zero measured hours lost. The hourly pipeline has carried it forward from there to the current 8,835 rows.

Raw columns. The stored schema comprises the UTC `timestamp`; eight pollutant concentrations (`co`, `no`, `no2`, `o3`, `so2`, `pm2_5`, `pm10`, `nh3`); six meteorological variables (`temp`, `humidity`, `pressure`, `wind_speed`, `wind_deg`, `clouds`); calendar fields; the computed `aqi` and provider `aqi_raw`; an `is_imputed` flag; and three lightweight derived columns persisted at ingestion time.

3.3 Feature Engineering

Persisting only raw observations keeps the store small and re-derivable; the full feature set is constructed on demand by `train_pipeline.engineer_features()`, which is called identically at training time and at inference time. This shared function is the single guarantee against train/serve skew. It expands the 27 stored columns into **64 model features** in six groups.

Table 2: Engineered feature groups.

Group	Construction	Rationale
Cyclical time	sin, cos of hour, day-of-week and month	Hour 23 and hour 0 are adjacent in reality but maximally distant as integers; sine/cosine pairs restore that adjacency. Karachi shows a strong diurnal pollution cycle.
AQI lags	$t - 1, 2, 3, 6, 12, 24, 48$ h	Air quality is highly autocorrelated. The 24 h and 48 h lags additionally encode same-time-yesterday behaviour.
Rolling statistics	mean, standard deviation, minimum, maximum over 3, 6, 12, 24 h windows	The mean gives a denoised level, the standard deviation gives local volatility, and min/max bracket the recent envelope.
Rates of change	first differences at 1, 3, 6, 24 h; deviation of current AQI from its 6 h and 24 h rolling means	Distinguishes a rising episode from a falling one at the same absolute level – the same AQI means very different things on the way up and on the way down.
Pollutant and weather lags	PM _{2.5} at 1, 3, 6, 24 h; wind speed and humidity at 6, 24 h	PM _{2.5} is the dominant AQI driver locally; yesterday’s wind and humidity condition today’s accumulation.
Interactions	wind speed $\times$ PM _{2.5} ; temp $\times$ humidity; 6 h pressure difference	Encodes dispersal (wind clears particulates), inversion conditions (warm and humid air traps pollution), and frontal passage.

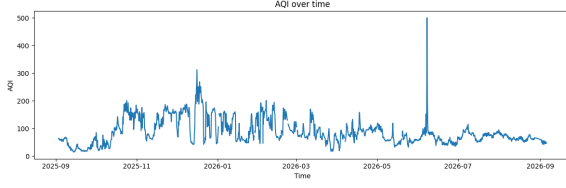
Two further design decisions in this stage are worth stating explicitly.

First, all lag and rolling operations are *positional* (**shift**, **rolling**), which is only correct on an unbroken hourly index. The store is therefore always passed through `utils.to_hourly_grid()` before feature construction, so that `shift(24)` genuinely means “24 hours ago” and not “24 records ago”.

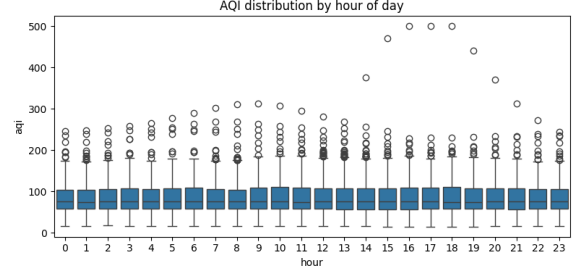
Second, AQI values above the 99th percentile are capped before lag and rolling features are derived. Karachi’s series contains short extreme spikes that otherwise propagate through 24 hours of rolling windows and dominate the fit.

3.4 Exploratory Data Analysis

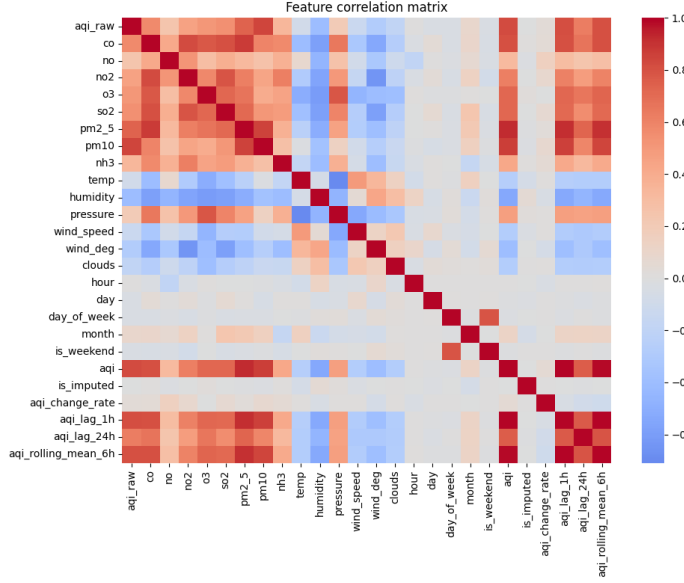
`notebooks/eda.py` regenerates three diagnostic figures from the live store (Figure 2). They establish the three properties the feature design relies on: a strong diurnal cycle, dominance of particulate matter over gaseous pollutants, and heavy autocorrelation with episodic spikes.



(a) AQI over the full coverage period.



(b) AQI distribution by hour of day.



(c) Correlation matrix of numeric features.

Figure 2: Exploratory analysis of the feature store. The time series shows sustained seasonal elevation with episodic spikes; the hourly distribution confirms a pronounced diurnal cycle that motivates the cyclical time encoding; the correlation matrix shows $PM_{2.5}$ and PM_{10} as the strongest correlates of AQI.

4 Feature Store

4.1 The Dual-Write Design

The feature store is written to two backends on every ingestion and read from one. This is a deliberate separation of the two things a feature store is normally asked to do at once.

Local Parquet – the source of truth. `data/features.parquet` is authoritative. Training, inference, the REST API and the dashboard all read from it and only from it. The reasons are operational: it has a single dependency (`pyarrow`), it loads in milliseconds, it works offline, and because it lives in the Git repository it inherits versioning, durability and point-in-time recovery for free. Critically for a serverless design, it means a Streamlit Community Cloud container can serve forecasts without holding credentials to any external system.

Hopworks – the managed production surface. In parallel, the same rows are written to a Hopworks feature group (`aqi_features`, version 2) configured with `timestamp` as both the primary key and the event time. This provides the capabilities a file-backed store cannot: a queryable online store, schema enforcement and lineage, feature statistics, and multi-consumer

access. Because `timestamp` is the primary key, an insert is an upsert, so re-sending an hour corrects it rather than duplicating it – the same idempotency property the Parquet merge has.

Fail-soft coupling. The dual-write is wrapped so that a Hopsworks failure is logged and swallowed: `utils._try_hopsworks_dual_write()` never propagates an exception into the pipeline. An outage or credential expiry at the managed backend degrades the system to Parquet-only operation rather than dropping an hour of data. `tests/test_feature_pipeline_local.py` asserts this behaviour directly: one test runs the pipeline with Hopsworks disabled, another forces a Hopsworks failure and asserts the Parquet write still succeeds.

4.2 Tuning the Ingestion Method for the Deployment Environment

One engineering decision in this layer is worth recording, because it shaped the final configuration.

Hopsworks offers two ingestion paths into a feature group. The default writer materialises data through a Delta/HDFS client, which expects a POSIX-like filesystem on the writing host. The alternative, enabled with `stream=True`, publishes rows to a Kafka topic and lets the Hopsworks cluster perform materialisation server-side.

Because this project’s pipeline must run both on a native Windows development machine and on a Linux GitHub Actions runner, the client-side writer was not a portable choice. The feature group was therefore provisioned as version 2 with `stream=True`, moving materialisation onto the managed cluster and reducing the client’s responsibility to publishing a well-typed record. This made the ingestion path identical in both environments, which is precisely the property a CI-driven pipeline needs. Version 1 of the feature group is retained but unused. The serving dependency set pins `hopsworks[python]==5.0.*` so that the Kafka client (`confluent-kafka`) required by stream ingestion is installed reproducibly in CI.

5 Model Training and Evaluation

5.1 Training Pipeline

`src/train_pipeline.py` executes the following sequence, once per forecast horizon $h \in \{24, 48, 72\}$:

1. **Load** the full feature store and reindex onto a continuous hourly grid.
2. **Engineer** the 64 features described in Table 2.
3. **Construct targets** by shifting the capped AQI backwards by h hours, so that row t carries the AQI observed at $t + h$. The final h rows have undefined targets and are dropped.
4. **Convert to a delta target** (Section 5.3).
5. **Cross-validate** four candidate families under `TimeSeriesSplit` with five expanding-window folds and a gap of h hours between train and test.
6. **Score** a persistence baseline on identical folds.
7. **Select** the candidate with the lowest pooled out-of-sample RMSE, refit it on 100% of the data, and write the bundle, metrics and SHAP artefacts to the registry.

5.2 Candidate Models

Four families compete at every horizon, spanning three modelling paradigms:

- **Ridge regression** ($\alpha = 10$) – a regularised linear baseline representing the classical statistical approach.
- **Random Forest** – 200 trees, maximum depth 12, minimum 5 samples per leaf, 60% of features per split. A bagged ensemble, so the spread across its trees is a usable variance estimate.
- **Gradient Boosting** – 200 stages, depth 4, learning rate 0.04, subsample 0.8.
- **Keras LSTM** – a recurrent network over 12-hour sequence windows, using a deliberately smaller set of 21 raw features. Feeding it the engineered lag columns would be redundant, since learning temporal structure from the sequence is precisely its job.

The LSTM is an optional candidate: if TensorFlow is unavailable it is skipped cleanly and the three scikit-learn families still train. The daily CI training job installs `requirements-train.txt` so that the LSTM does compete in automated retraining, while the deployed serving environment omits TensorFlow entirely.

5.3 Predicting the Change Rather than the Level

All models predict $\Delta\text{AQI} = \text{AQI}(t+h) - \text{AQI}(t)$, the change from the current reading, rather than the absolute future level. The reconstruction at inference time is simply $\widehat{\text{AQI}}(t+h) = \text{AQI}(t) + \widehat{\Delta}$, clipped to $[0, 500]$.

The motivation is structural. With roughly one year of history, the earliest expanding-window fold trains on late-summer air and is scored on winter smog: per-fold target means in the recorded metrics range from 65.3 to 133.7 AQI points across folds. Tree ensembles cannot extrapolate beyond their training range, so predicting the level directly yields a negative R^2 on that fold regardless of feature quality. The hour-to-hour *change* is far closer to stationary across seasons, so the same fitted model transfers between them.

5.4 Evaluation Protocol

Evaluation uses `TimeSeriesSplit` with five expanding-window folds and an explicit gap of h hours between the end of each training window and the start of its test window. The gap matters: without it, a row whose target lies h hours in the future would overlap the test period, leaking the answer. Three metrics are recorded – RMSE (the selection criterion, penalising the large errors that matter most for hazard warnings), MAE (an interpretable typical error in AQI points), and R^2 (variance explained).

Every horizon is additionally scored against a **persistence baseline**: “the AQI in h hours will be what it is now.” This is a demanding reference for a highly autocorrelated series, and it is the honest bar a forecast must clear to be worth publishing. The verdict is stored as `beats_baseline` in each metrics file, and `predict.py` reads it at serving time: if a horizon’s model failed to beat persistence, the dashboard serves persistence for that horizon rather than knowingly publishing the worse forecast. In the current registry all three horizons beat the baseline, so all three serve their trained model.

5.5 Results

Models were trained on 2026-09-05 between 16:48 and 16:54 UTC with scikit-learn 1.9.0, on 8,176, 8,164 and 8,140 usable rows for the 24h, 48h and 72h horizons respectively. Table 3 gives the pooled out-of-sample metrics for every candidate at every horizon.

Table 3: Pooled out-of-sample metrics from five-fold expanding-window time-series cross-validation. Best RMSE per horizon in **bold**. All values in AQI points except R^2 .

Horizon	Model	RMSE	MAE	R^2
+24 h	Ridge regression	32.38	21.93	0.397
	Random Forest	27.43	17.31	0.567
	Gradient Boosting	28.36	18.08	0.537
	LSTM	29.04	17.44	0.508
	<i>Persistence baseline</i>	29.51	17.46	0.499
+48 h	Ridge regression	43.23	30.96	-0.079
	Random Forest	34.98	24.49	0.294
	Gradient Boosting	36.34	26.06	0.237
	LSTM	38.36	24.87	0.145
	<i>Persistence baseline</i>	39.09	25.10	0.118
+72 h	Ridge regression	48.28	35.58	-0.366
	Random Forest	39.14	28.24	0.102
	Gradient Boosting	41.66	30.30	-0.017
	LSTM	41.76	28.35	-0.021
	<i>Persistence baseline</i>	42.57	28.31	-0.062

Deployment decision. **Random Forest is deployed for all three horizons.** It attained the lowest RMSE of any candidate at every horizon, and in each case beat the persistence baseline, so `beats_baseline` is `true` throughout and each horizon serves its trained model rather than falling back to persistence. Table 4 quantifies the margin.

Table 4: Deployed model versus persistence baseline.

Horizon	Deployed	Model RMSE	Baseline RMSE	RMSE reduction	<code>beats_baseline</code>
+24 h	Random Forest	27.43	29.51	7.0%	<code>true</code>
+48 h	Random Forest	34.98	39.09	10.5%	<code>true</code>
+72 h	Random Forest	39.14	42.57	8.1%	<code>true</code>

Three observations follow from these numbers.

The **ordering of families is consistent and interpretable**. Random Forest wins everywhere, Gradient Boosting is a close second, the LSTM is competitive on MAE but not RMSE, and Ridge is last – decisively so at longer horizons, where its R^2 turns negative. With a few thousand training rows, this is the expected ranking: bagged trees handle the non-linear, interacting, moderately-sized tabular problem better than either a linear model or a deep sequence model.

Skill degrades with horizon, as it must. R^2 falls from 0.567 at +24 h to 0.294 at +48 h to 0.102 at +72 h, and RMSE rises from 27.4 to 39.1 AQI points. AQI three days out depends on weather that is itself only forecastable with uncertainty, plus traffic, industrial activity and unforecastable one-off events such as dust storms. Reporting this degradation rather than obscuring it is part of the result.

The relative gain over persistence is largest in the middle. At +24 h persistence is very strong, because air quality one day out is heavily constrained by air quality now, leaving the model only 7.0% to gain. At +48 h persistence weakens faster than the model does, opening the widest margin at 10.5%.

A note on interpreting these figures. The metrics are out-of-sample estimates from cross-validation, while the deployed artefact is refit on 100% of the data – so they estimate its generalisation rather than measure it. Per-fold results are also retained in each metrics file,

because the earliest expanding-window fold trains on a single season and is scored on another and therefore dominates the pooled figures. At +24 h, for example, per-fold RMSE ranges from 39.92 on the earliest fold down to 6.30 on the most recent, the latter being the fold whose training window most resembles current serving conditions.

6 Explainability with SHAP

6.1 Why Explainability Matters Here

A 200-tree Random Forest is not inspectable by reading it. For a public-health forecast that is a real problem, for three reasons.

Trust. A user asked to change their behaviour on the strength of a prediction is entitled to know what drove it. “Unhealthy air expected in 48 hours” is far more actionable when accompanied by “because particulate levels are already elevated and pressure is falling.”

Debugging. Feature attributions catch silent failures that aggregate metrics hide. If a model’s dominant driver were a calendar artefact rather than a physical quantity, RMSE alone would not reveal it, but a SHAP summary would immediately.

Scientific plausibility. Where attributions agree with atmospheric chemistry – particulates, pressure, recent AQI trend – the model has likely learned mechanism rather than coincidence.

6.2 Methodology

SHAP (SHapley Additive exPlanations) assigns each feature a contribution to an individual prediction, derived from cooperative game theory, such that the contributions sum exactly to the difference between that prediction and the dataset’s mean prediction. It is therefore both locally faithful and globally aggregable.

Because every deployed model is a tree ensemble, **TreeExplainer** is used, which computes exact Shapley values for tree models rather than approximating them. The pipeline runs SHAP *per horizon, on the deployed model for that horizon*, over a sample of 100 representative rows, and writes two artefacts: a beeswarm summary plot (`shap_summary_{h}h.png`) and a ranked machine-readable importance file (`shap_importance_{h}h.json`) holding mean |SHAP| per feature so the dashboard can display drivers without recomputing anything. The current artefacts were generated on 2026-09-05 at 21:56 UTC. `predict.py` also timestamp-compares each plot against its model and flags it as stale if the model is newer, so the dashboard can never present an explanation belonging to a superseded model.

In the beeswarm plots that follow, features are ordered top-to-bottom by mean absolute SHAP value. Each point is one sample; horizontal position is that feature’s signed contribution to the predicted AQI change, and colour encodes whether the feature’s own value was high or low.

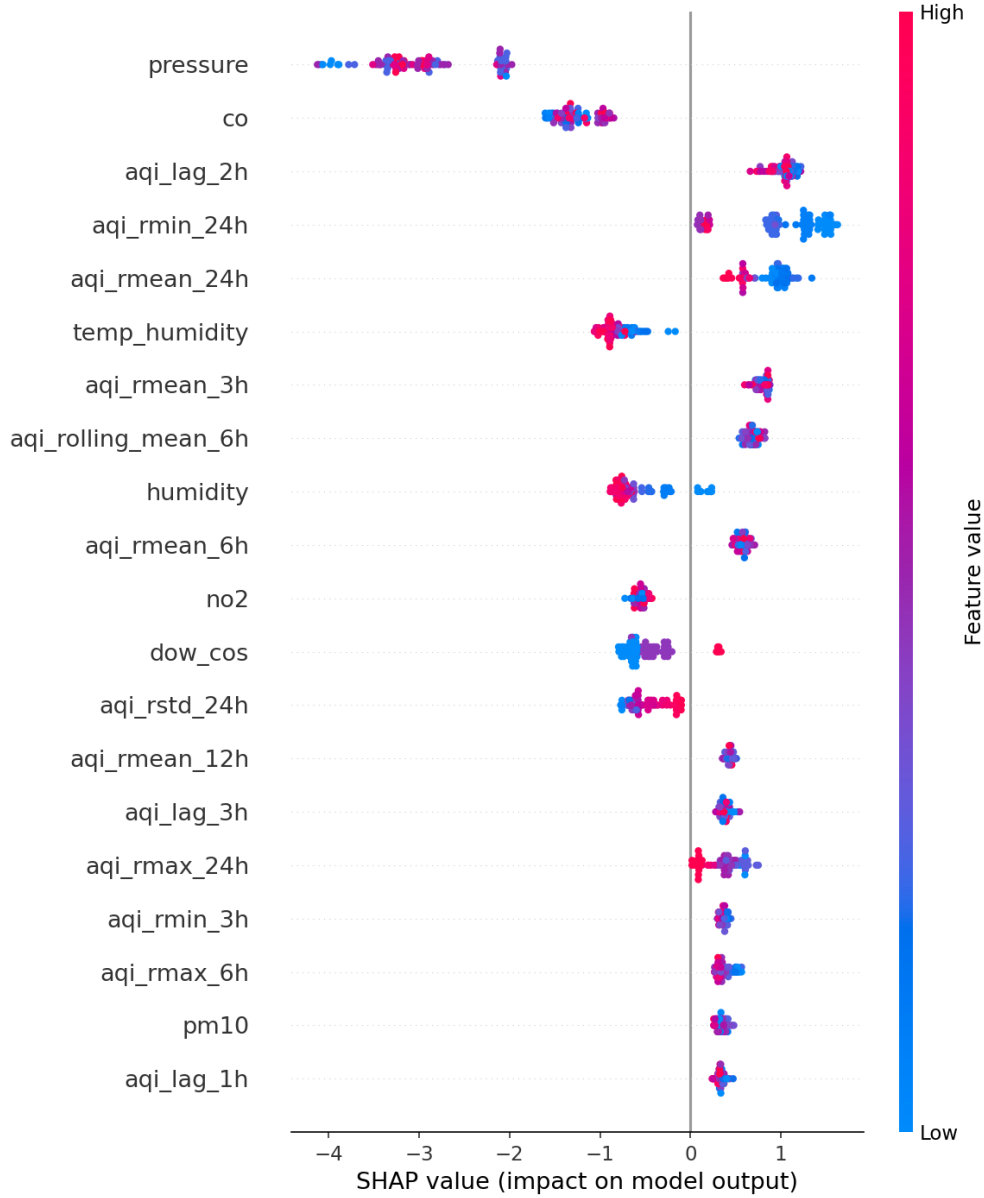


Figure 3: SHAP summary for the deployed +24 h Random Forest. Atmospheric **pressure** is the single strongest driver (mean $|\text{SHAP}| = 2.96$ AQI points), consistent with pressure acting as a proxy for the dispersal-versus-inversion regime that determines whether pollutants accumulate. **Carbon monoxide** follows at 1.25, a direct combustion and traffic tracer. The remainder of the top ranks are dominated by recent AQI history – the 2-hour lag (1.04), the 24-hour rolling minimum (0.98) and the 24-hour rolling mean (0.87) – confirming that at one day out the model leans heavily on persistence-like information, exactly as the narrow 7.0% margin over the persistence baseline would predict.

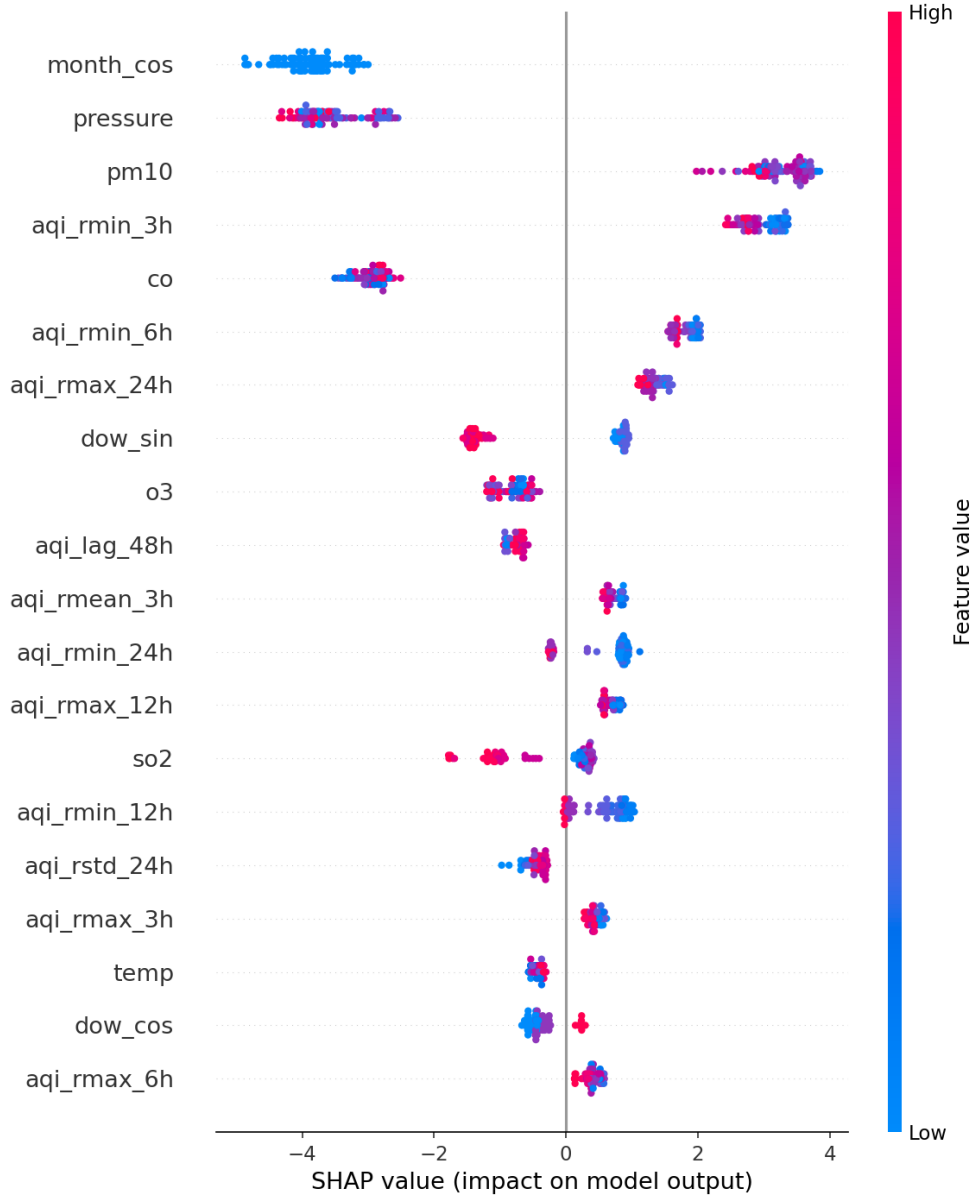


Figure 4: SHAP summary for the deployed +48h Random Forest. The driver set shifts materially. **Seasonal phase** (`month_cos`, 3.88) becomes the leading feature, **pressure** remains strong at 3.53, and **coarse particulate matter** (`PM10`, 3.24) enters the top three alongside **carbon monoxide** (2.96). Short-window AQI statistics such as the 3-hour and 6-hour rolling minima are still present but have been displaced from the top. This is the expected transition: two days out, the immediate past carries less information than the physical and seasonal state of the atmosphere.

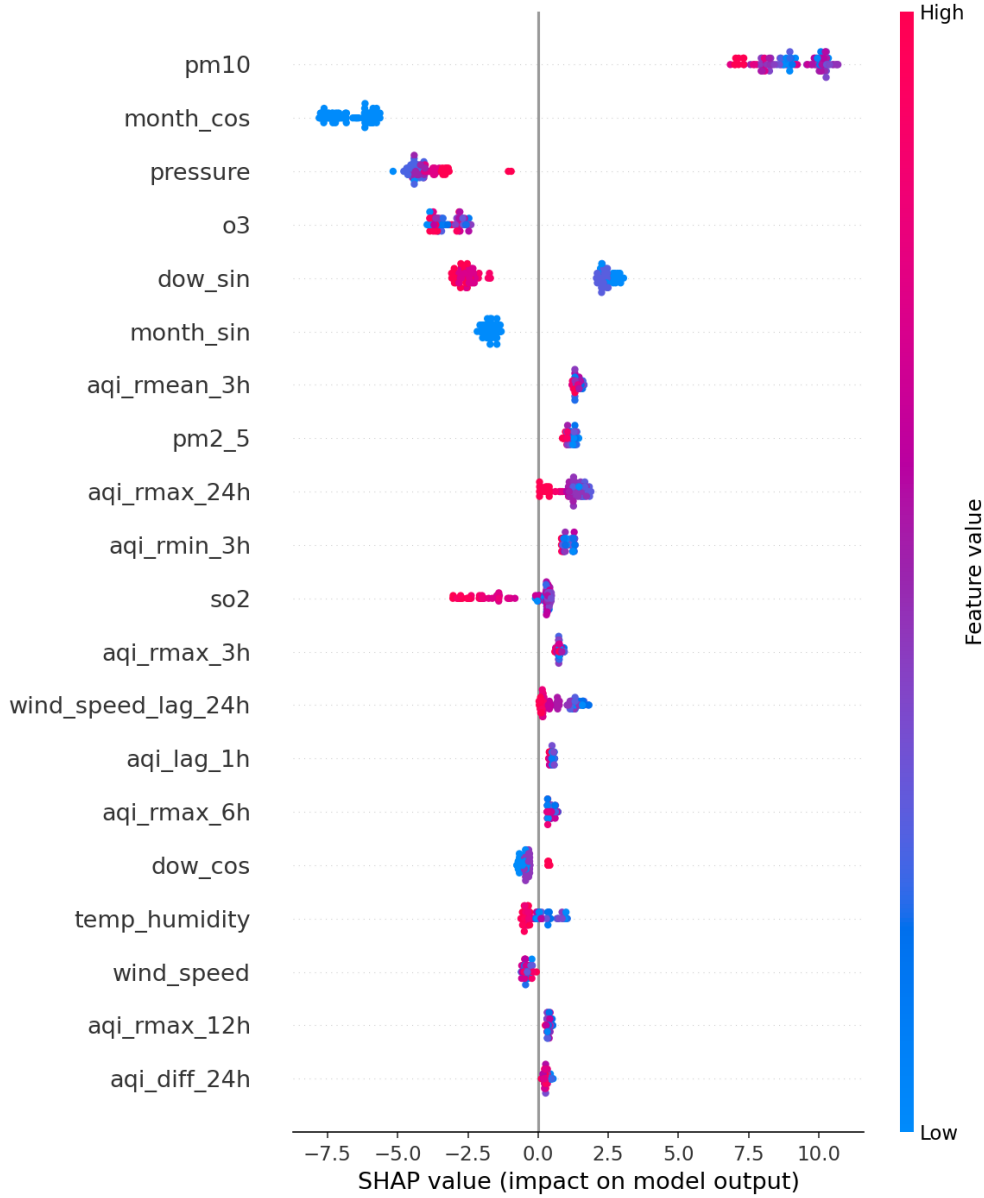


Figure 5: SHAP summary for the deployed +72 h Random Forest. The shift away from recent history completes. **PM₁₀** dominates at 9.05 mean |SHAP| – roughly three times the magnitude of any driver at +24 h – followed by **seasonal phase** (6.60), **pressure** (4.07) and **ozone** (3.23). PM_{2.5} appears at 1.19 and the strongest recent-history term, the 1-hour AQI lag, has fallen to 0.49. At three days out the model is effectively predicting a climatology-plus-current-loading regime rather than extrapolating a trend, which is consistent with the low R^2 of 0.102 at this horizon.

Interpretation across horizons. Read together, the three figures tell a coherent physical story. As the horizon lengthens, attribution migrates from *recent AQI dynamics* (lags and short rolling windows at +24 h) to *atmospheric and seasonal state* (PM₁₀ loading, pressure, month, ozone at +72 h). This is exactly the behaviour a well-specified forecasting model should exhibit: short-range skill comes from momentum, long-range skill comes from regime.

7 Prediction Mechanism

Both serving surfaces call the same function, `predict.get_forecast()`. The following six steps describe precisely what happens when a user opens the dashboard or issues `GET /forecast`.

Step 1 – Load and prepare history.

`_prepared_history()` reads the entire Parquet feature store, reindexes it onto a continuous hourly grid via `to_hourly_grid()`, and applies `engineer_features()`. Using the same function as training is what eliminates train/serve skew. The forecast is then anchored to the most recent row that carries an *actually measured* AQI, not merely the last row on the grid – a grid can legitimately end on placeholder hours that were never observed. That anchor row's capped AQI becomes $AQI(t)$, which is simultaneously the model's reference point and what the persistence baseline would forecast.

Step 2 – Compute lag and rolling features.

Because the anchor is a row of the fully engineered frame, its 64 features already include every lag, rolling statistic, rate of change and interaction, computed from the preceding history rather than requested from the model. Before use, the vector is validated: if any required column is missing, or any value is null – the 48-hour lag is undefined until the store holds that much history – that horizon is skipped with an explicit, human-readable reason rather than silently producing a wrong number.

Step 3 – Scale with the deployed model's own transformer.

Each registry bundle contains the `StandardScaler` that was fitted during that model's final refit, alongside the estimator itself and the exact ordered list of feature columns. The anchor vector is projected onto that column order and transformed by that scaler, so the numbers the model sees at serving time are distributed exactly as they were at training time. For a sequence model the last `seq_len` consecutive hours are stacked into a single batch instead of a single row.

Step 4 – Predict the change and reconstruct the level.

The estimator outputs $\hat{\Delta}$, the predicted change from the anchor. `predict_with_interval()` reconstructs the absolute forecast as $\widehat{AQI}(t+h) = AQI(t) + \hat{\Delta}$ and clips it to the valid EPA range $[0, 500]$. This runs once per horizon, producing the +24 h, +48 h and +72 h forecasts.

Step 5 – Derive an uncertainty band.

The band is anchored to `residual_std`, the standard deviation of the model's own out-of-sample cross-validation errors – currently 27.36, 34.75 and 38.79 AQI points for the three horizons. This captures total error, not merely model variance. For a Random Forest the width is then modulated by how much the individual trees disagree about *this particular input*: the per-input spread is divided by the training-set mean spread and clamped to $[0.5, 2.0]$, so unusual conditions yield visibly wider bands while a single odd input cannot produce an absurd one. The half-width is $z\sigma$ with $z = 1.2816$, a central 80% interval. The dashboard labels this as a heuristic band rather than a calibrated prediction interval, which it is.

Step 6 – Return a uniform result.

Each horizon yields the forecast timestamp, point prediction, lower and upper bounds, EPA category, the serving method, the model name, and the `beats_baseline` verdict. If that verdict is false for a horizon, `predict.py` substitutes the persistence forecast with a band derived from the *baseline's* residual standard deviation, and labels the method "`persistence`" so the substitution is visible rather than hidden. The resulting structure is what FastAPI serialises into its response model and what Streamlit renders into forecast cards. **Neither surface contains any prediction logic of its own**, so the API and the dashboard are guaranteed to report identical numbers.

8 Automation and CI/CD

8.1 Workflow Architecture

Two GitHub Actions workflows constitute the entire operational scheduler.

Table 5: Scheduled workflows.

Workflow	Schedule	Actions
feature_pipeline.yml	0 * * * * (hourly)	Checkout, Python 3.11 with pip cache, install <code>requirements.txt</code> , run <code>src/feature_pipeline.py</code> with the OpenWeather secret and location variables injected, then commit and push <code>data/features.parquet</code> if it changed.
training_pipeline.yml	0 3 * * * (daily, 03:00 UTC)	Checkout, Python 3.11, install <code>requirements.txt</code> and <code>requirements-train.txt</code> (adding TensorFlow so the LSTM competes), run <code>src/train_pipeline.py</code> , then commit and push the refreshed <code>models/</code> directory. Capped at 90 minutes.

Both workflows also expose `workflow_dispatch` for manual triggering, and both are hardened in three ways that matter in practice:

- **permissions: contents: write.** The default `GITHUB_TOKEN` is read-only on new repositories, so the commit-back step would fail with HTTP 403 without this grant.
- **A shared concurrency group** (`aqi-repo-commit`). Both jobs commit to the same branch and would otherwise collide at 03:00 UTC, when their schedules coincide. Serialising them eliminates non-fast-forward rejections by construction.
- **A rebase-and-retry push loop.** Each job attempts up to three pushes, rebasing onto `origin` with `autostash` between attempts, so a concurrent push cannot lose an hour of data. Commits are tagged `[skip ci]` to prevent a self-triggering loop.

Both jobs also guard against the legitimate first-run state in which no artefacts exist yet – `git add` on a missing path is an error, so the step exits cleanly instead.

8.2 Evidence of Independent Scheduled Execution

The automation is not aspirational; it has been running unattended. The repository’s commit history records **151 commits authored by the aqi-bot CI identity**, none of them initiated by a human:

- **146 hourly “Update features” commits** from the feature pipeline, beginning 2026-09-01 10:33:18 UTC and continuing to 2026-09-06 02:05:27 UTC.
- **5 daily “Retrain models” commits** from the training pipeline, at 2026-09-01 10:42:51, 2026-09-02 07:30:37, 2026-09-03 07:35:20, 2026-09-04 07:35:40 and 2026-09-05 07:18:01 UTC.

The hourly cadence is visible directly in consecutive timestamps – for example 2026-09-05 at 20:01:17, 21:01:18, 22:01:13 and 23:01:18 UTC, then 2026-09-06 at 00:01:26 and 01:01:21 UTC. The retrain commits land on their own daily schedule, independent of the feature commits, confirming that the two workflows execute separately. Together these constitute a five-day unbroken record of a serverless system collecting data and retraining itself with no manual intervention.

9 Web Application

9.1 Streamlit Dashboard

The human interface is a two-page Streamlit application (`app/streamlit_app.py`) with a shared theme (`app/theme.py`, palette in `.streamlit/config.toml`) and a cached data-access layer (`app/data_access.py`) that wraps `predict.py` behind `@st.cache_data` so a page interaction does not re-read the store.

Page 1 – Dashboard (`app/page_dashboard.py`). The operational view, designed to answer “what is the air like and what is it about to do” without scrolling:

- A **current AQI gauge**, colour-coded to the EPA category scale, with the observation timestamp and a staleness warning if the anchor row is more than six hours old.
- **Three forecast cards** for +24 h, +48 h and +72 h, each showing the point forecast, its EPA category and colour, and the uncertainty band.
- A prominent **alert banner** when the hazard threshold is reached (Section 10).
- A **key pollutant grid** giving current concentrations for PM_{2.5}, PM₁₀, O₃, NO₂, SO₂ and CO, plus the identified dominant pollutant.
- A **recent-trend sparkline** and the **top SHAP drivers** for the active forecast.

Page 2 – Model Analysis (`app/page_analysis.py`). The transparency view, for a user who wants to know whether to believe the forecast:

- A **14-day historical AQI chart** with forecast points and uncertainty bands overlaid.
- A **model accuracy comparison** rendered live from the registry’s `metrics_*.json` – RMSE, MAE and R^2 per horizon against the persistence baseline, so the numbers in Table 3 are visible in the product itself.
- A **SHAP explainability panel** showing the summary plot and ranked feature importances per horizon, with the staleness flag from `predict.shap_plot_status()` surfaced to the user.

9.2 FastAPI Backend

`api/main.py` exposes the same capabilities as a machine interface, with Pydantic response models and auto-generated OpenAPI documentation at `/docs`.

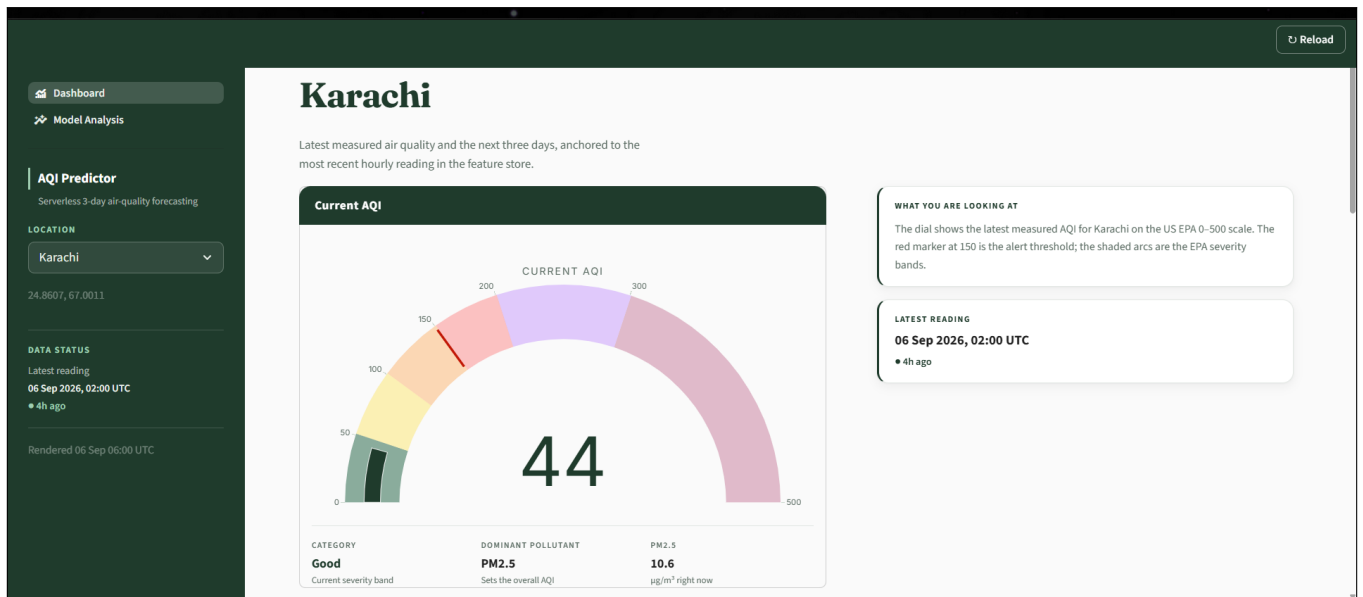
Table 6: REST API endpoints.

Endpoint	Purpose
GET /health	Liveness probe: feature store row count and which horizons currently have loadable models.
GET /current	Latest measured AQI, EPA category, dominant pollutant, observation age and staleness flag.
GET /forecast	Three-day forecast with uncertainty bands, serving method and baseline verdict per horizon.
GET /history?hours=	Recent observed AQI series (default 336 hours, i.e. 14 days).
GET /alerts	Whether hazardous air is currently measured or forecast, with the triggering horizon and a human-readable message.
GET /metrics	Per-horizon model metrics and baseline comparison, read from the registry.
GET /categories	The full EPA category table with alerting flags.
GET /categories/{aqi}	Classify an arbitrary AQI value.

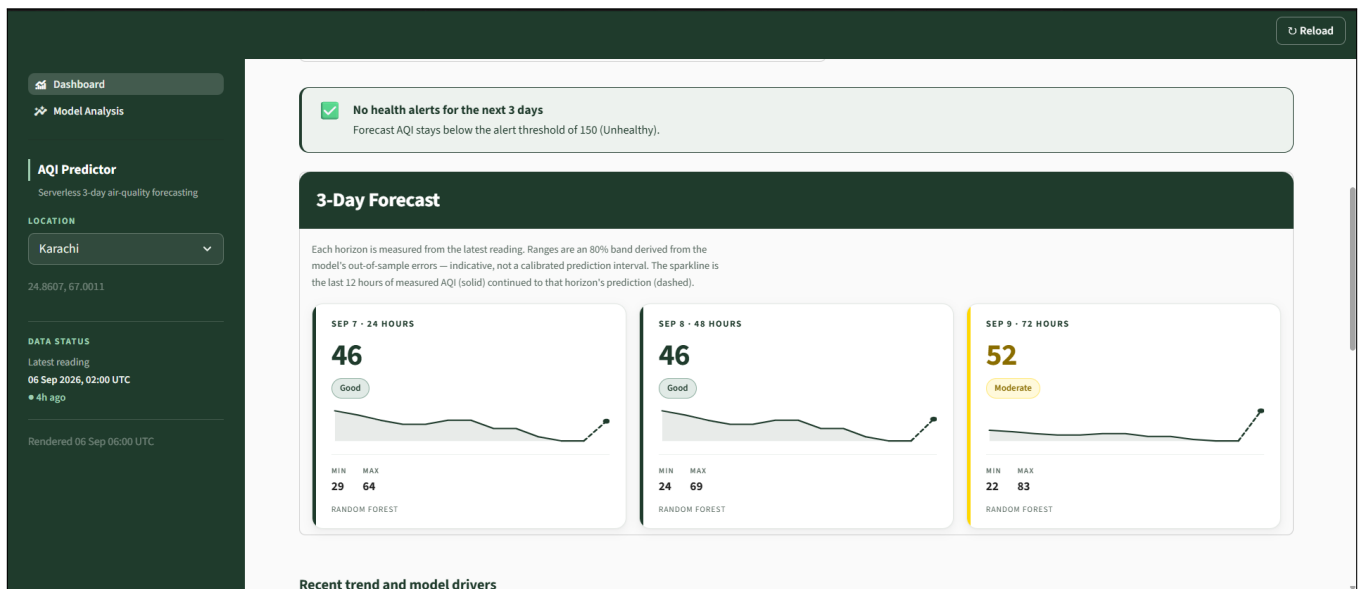
Because both surfaces delegate to `predict.py`, the dashboard and the API are structurally incapable of reporting different forecasts.

9.3 Deployed Interface

Figures 6 and 7 show the live application, captured on 6 September 2026 against the feature store and model registry described throughout this report. Each page is presented as a two-panel scroll capture, since both are taller than a single browser viewport.

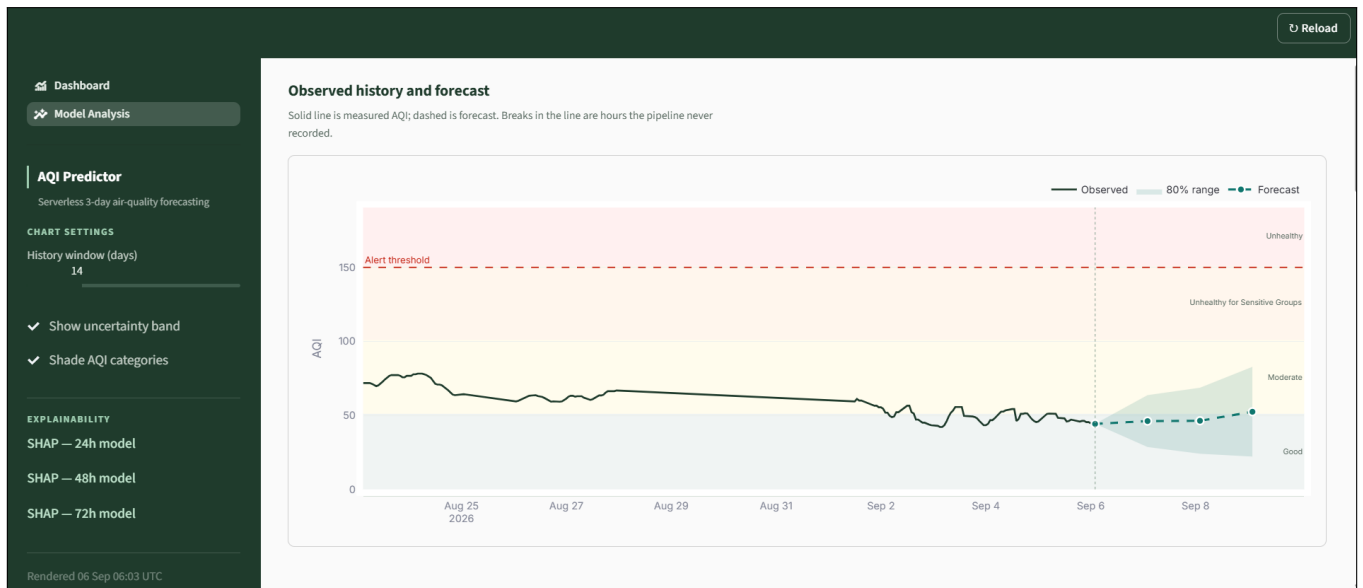


(a) Upper panel: current-conditions hero and AQI gauge.

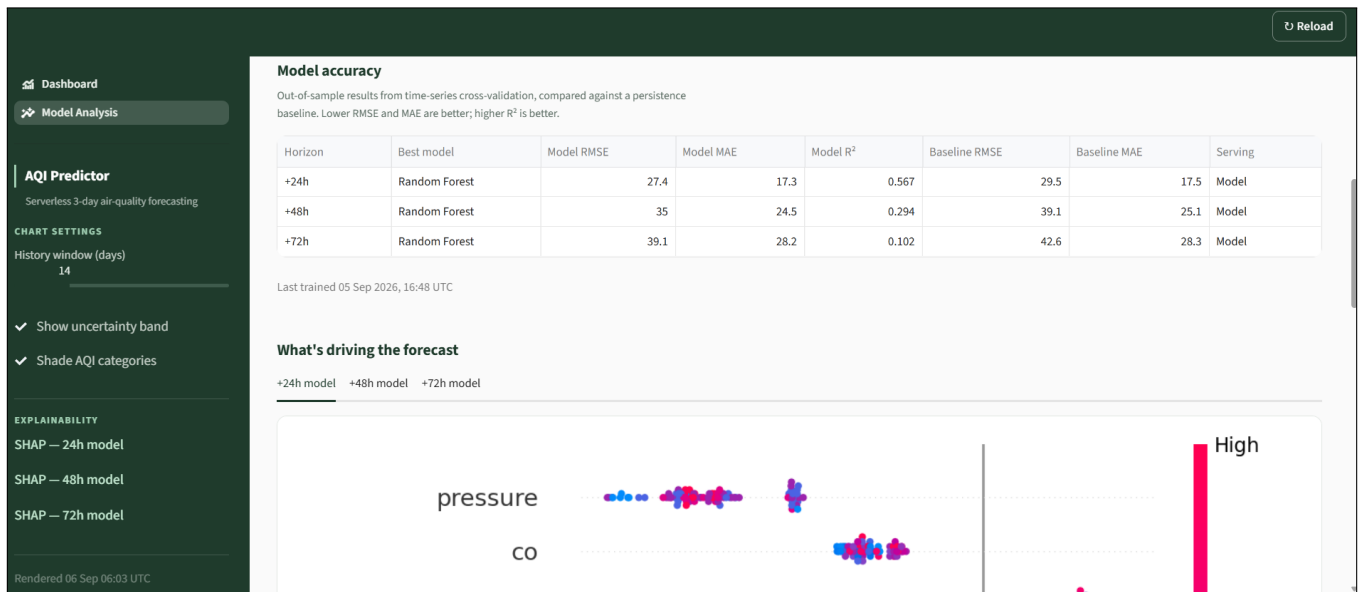


(b) Lower panel: alert state and three-day forecast cards.

Figure 6: Dashboard page, full view of the deployed Streamlit application. (a) The gauge renders the latest measured AQI of 44 on the EPA 0–500 scale, with the shaded arcs marking the severity bands and the red marker fixed at the alert threshold of 150. Beneath it the page reports the category (*Good*), the dominant pollutant driving the index (PM_{2.5}) and its concentration (10.6 µg m⁻³). The sidebar carries the location selector, the coordinates, and a data-status block showing the reading is four hours old. (b) The alert region is in its non-triggered state – “No health alerts for the next 3 days” – because the forecast stays below 150. The three forecast cards give 46 (*Good*) at +24 h, 46 (*Good*) at +48 h and 52 (*Moderate*) at +72 h, each with its 80% band (29–64, 24–69 and 22–83 respectively) and each labelled RANDOM FOREST, matching the deployment decision recorded in Table 4. Note how the bands widen with horizon, a direct visual consequence of the growing residual standard deviations reported in Section 5.3.



(a) Upper panel: observed history with forecast and uncertainty band.



(b) Lower panel: model accuracy table and SHAP explainability panel.

Figure 7: Model Analysis page, full view of the deployed application. (a) Fourteen days of measured AQI (solid line) continue into the three-day forecast (dashed) with its shaded 80% range. The background is shaded by EPA category and the red dashed rule marks the alert threshold at 150, placing the current regime visually well inside the *Good* and *Moderate* bands. The sidebar exposes the history window, uncertainty-band and category-shading toggles, and per-horizon SHAP links. (b) The accuracy table is rendered live from `models/metrics_*.json` and reproduces exactly the figures in Table 3 – RMSE 27.4, MAE 17.3 and R^2 0.567 at +24h against a baseline RMSE of 29.5, and so on for the longer horizons – with a **Serving** column confirming all three horizons serve the trained model rather than the baseline. The footer records the registry’s training time of 05 Sep 2026, 16:48 UTC. Below, the “What’s driving the forecast” panel shows the SHAP beeswarm for the selected horizon, with **pressure** and **co** as the leading +24 h drivers, agreeing with Figure 3.

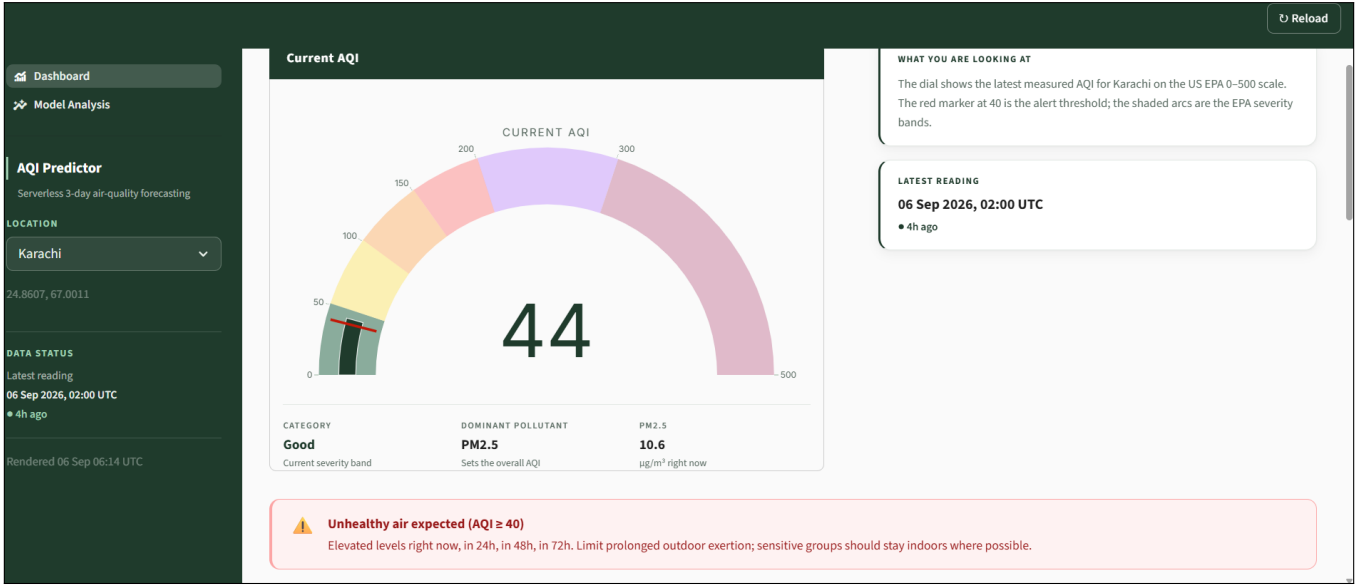


Figure 8: Dashboard page, alert-triggered state, showing the red hazard banner and the gauge’s threshold marker. The banner names every offending horizon – “Elevated levels right now, in 24h, in 48h, in 72h” – and pairs it with the public-health advice to limit prolonged outdoor exertion. Karachi’s air remained in the *Good* to *Moderate* range throughout the capture window (measured AQI 44.2; forecasts 46.1, 46.3 and 52.4), so no genuine exceedance of the production threshold was available to photograph. The threshold was therefore lowered temporarily purely to exercise the alerting path, which is why the banner text and the gauge marker read a reduced value rather than 150. The alerting *logic* shown is exactly the deployed logic; only the constant differs, and the production value of `ALERT_THRESHOLD = 150` was restored immediately afterwards – Figure 6a shows the same gauge with the marker back at 150.

10 Alerts System

10.1 Threshold Logic

Alerting is grounded in the published US EPA category scale rather than an arbitrary cutoff. `config.py` encodes the six official bands as half-open intervals, $lo \leq AQI < hi$, so that a fractional value such as 50.4 cannot fall between two categories; the final band is closed at the top and absorbs anything above 500 (Table 7).

Table 7: EPA AQI categories as implemented, with alerting status.

AQI range	Category	Triggers alert
$0 \leq AQI < 51$	Good	
$51 \leq AQI < 101$	Moderate	
$101 \leq AQI < 151$	Unhealthy for Sensitive Groups	
$151 \leq AQI < 201$	Unhealthy	✓
$201 \leq AQI < 301$	Very Unhealthy	✓
$301 \leq AQI \leq 501$	Hazardous	✓

The alert threshold is set at `ALERT_THRESHOLD = 150`, the entry point of the EPA’s *Unhealthy* classification. This is the correct trigger point for a public-health warning: below it, EPA guidance concerns sensitive groups only, whereas at and above it the agency advises that members of the *general* population may begin to experience adverse effects. Anchoring the threshold to a published standard rather than a hand-tuned number means the alert carries an externally-defined meaning.

10.2 Surfacing the Alert

An alert fires if *either* the latest measured AQI *or* any of the three forecast values reaches the threshold. Alerting on the forecast, not only on the observation, is the entire point of the system – a warning that arrives after the episode has begun has no preventive value.

The condition is evaluated in two places against the same threshold constant:

- **In the dashboard**, as a prominent full-width banner directly beneath the gauge on the Dashboard page. The banner is always rendered in one of four fixed severity treatments, so the state is legible before any text is read: a red **danger** style when the threshold is reached (Figure 8) and a green **ok** style confirming “No health alerts for the next 3 days” when it is not (Figure 6b). The message names every offending horizon and adds the corresponding public-health advice.
- **Through the API**, at `GET /alerts`, which returns a boolean trigger flag, the threshold in force, the horizon responsible, and a human-readable message – either “Unhealthy air (AQI ≥ 150) expected ...” naming the offending horizon, or a confirmation that no reading at or above the threshold is measured or forecast. The same flag is echoed in the `/forecast` response, and `/categories` annotates every band with whether it alerts, so a downstream consumer can reproduce the logic exactly.

11 Deployment

The dashboard runs live on **Streamlit Community Cloud**, bound directly to the GitHub repository. There is no build server, container registry or deployment script: Streamlit Cloud watches the `main` branch, and any push – whether a human code change or an `aqi-bot` commit from the hourly feature pipeline – triggers a rebuild and redeploy. Continuous deployment is therefore a property of the hosting arrangement rather than something the project implements.

This closes the serverless loop. GitHub Actions collects data and retrains models, committing both back to the repository; Streamlit Cloud redeploys from that same repository; and the deployed app reads the feature store and model registry straight out of its own checkout. No component of the running system requires a machine that anyone administers.

Keeping the serving environment lightweight. Dependencies are split deliberately across two files so that training weight never reaches production:

- `requirements.txt` is the **serving stack** – pandas, NumPy, scikit-learn, joblib, pyarrow, Streamlit, Plotly, FastAPI, Uvicorn, Matplotlib, SHAP and the Hopsworks client. This is the only file Streamlit Cloud installs.
- `requirements-train.txt` adds `tensorflow-cpu`, and is installed *only* by the daily training workflow and by developers who intend to retrain.

The split is safe precisely because the deployed models are Random Forest joblibs at every horizon: `predict.py` imports TensorFlow lazily and only when a bundle is flagged as a sequence model, so the serving path never touches it. Excluding a multi-hundred-megabyte dependency keeps Streamlit Cloud builds inside their resource limits and materially reduces cold-start time.

Reproducibility measures. `runtime.txt` pins Streamlit Cloud to Python 3.11, matching the CI runners. scikit-learn is pinned to an exact version (1.9.0) rather than a range, because a joblib artefact records the scikit-learn version that produced it: allowing CI and local environments to drift across minor versions produced `InconsistentVersionWarning` on load. Other libraries carry lower bounds for reproducibility and upper bounds on the majors known to break

this code, and `hopsworks[python]` is pinned to the connected backend’s minor series (5.0.x). Finally, Keras artefact paths are always reconstructed locally by `config.keras_model_path()` rather than read from the bundle, so a CI absolute path such as `/home/runner/work/...` can never break a load on another machine.

12 Results Summary

Table 8 maps each original project requirement to its implementation and to the concrete evidence of fulfilment.

Table 8: Requirement fulfilment matrix.

Requirement	Implementation	Evidence of fulfilment
Feature pipeline	<code>src/feature_pipeline.py</code>	Runs hourly on GitHub Actions; fetches OpenWeather pollution and weather, computes EPA AQI from per-pollutant sub-indices, merges idempotently into the store. 146 automated feature commits recorded.
Historical back-fill	<code>src/backfill.py</code>	8,835 hourly rows spanning 368 days (2025-09-03 to 2026-09-06), 100% measured with no imputation and no gap exceeding one hour. Full lineage in <code>data/PROVENANCE.md</code> with SHA-256 checksums and retained raw API responses.
Feature store	<code>data/features.parquet</code> + Hopsworks <code>aqi_features v2</code>	Dual-write design: Parquet as dependency-free source of truth, managed Hopsworks feature group via Kafka stream ingestion with <code>timestamp</code> as primary key and event time. Fail-soft coupling verified by unit tests.
Training pipeline with RMSE / MAE / R^2	<code>src/train_pipeline.py</code>	Four candidate families $\times$ three horizons under five-fold expanding-window <code>TimeSeriesSplit</code> with horizon-length gap. All twelve candidate results plus per-fold breakdowns recorded in <code>models/metrics_*.json</code> (Table 3).
Model registry and automatic selection	<code>models/</code>	Best model per horizon selected by pooled out-of-sample RMSE, refit on all data, serialised with its scaler, feature list and residual statistics. Random Forest deployed at all three horizons.
Baseline comparison	<code>beats_baseline</code> in registry	Persistence baseline scored on identical folds. RMSE reduced by 7.0% (+24 h), 10.5% (+48 h) and 8.1% (+72 h); all three verdicts <code>true</code> , so all three serve their trained model (Table 4).

(Table 8 continued)

Requirement	Implementation	Evidence of fulfilment
CI/CD automation	<code>.github/workflows/</code>	Hourly feature workflow and daily 03:00 UTC training workflow, both serverless, with shared concurrency locking and rebase-retry pushes. 151 autonomous <code>aqi-bot</code> commits over five days.
Interactive dashboard with forecast and alerts	<code>app/</code>	Two-page Streamlit app: gauge, three forecast cards with uncertainty bands, pollutant grid, trend sparkline, and historical/accuracy/SHAP analysis page. Live on Streamlit Community Cloud with continuous deployment.
REST API	<code>api/main.py</code>	Eight FastAPI endpoints with Pydantic models and OpenAPI docs, sharing <code>predict.py</code> with the dashboard so the two cannot diverge.
Exploratory data analysis	<code>notebooks/eda.py</code>	Three regenerable figures in <code>data/eda/</code> establishing diurnal seasonality, particulate dominance and autocorrelation structure (Figure 2).
Explainability	SHAP TreeExplainer	Per-horizon summary plots and ranked importance JSON regenerated each training run, displayed in the dashboard with automatic staleness detection (Figures 3–5).
Alerts	<code>ALERT_THRESHOLD = 150</code>	EPA <i>Unhealthy</i> threshold applied to both measured and forecast AQI; surfaced as a colour-coded dashboard banner and via <code>GET /alerts</code> .
Uncertainty quantification	<code>predict_with_interval()</code>	Central 80% bands ($z = 1.2816$) from out-of-sample residual standard deviations of 27.36, 34.75 and 38.79 AQI points, modulated per input by Random Forest tree disagreement.

13 Limitations and Future Work

Single-location scope. The system currently forecasts for Karachi alone. This is a configuration choice, not an architectural constraint: location is read from environment variables (`AQI_CITY`, `AQI_LAT`, `AQI_LON`) and `config.py` already carries presets for Lahore, Islamabad, Peshawar, Quetta and Delhi. Extending to multiple cities means partitioning the feature store by location and training per-location models; the ingestion, training, serving and automation layers need no redesign. A shared multi-city model with location as a feature is the more interesting version, since it would let sparse-history cities borrow strength from dense ones.

Forecast weather as an input. This is the single highest-leverage enhancement available. The models currently condition on *current-time* features – present pollutant loading, present weather, and the recent history of both – and must implicitly infer the atmospheric evolution of the next one to three days. Yet that evolution is partly knowable: both OpenWeather and

Open-Meteo expose free forecast endpoints. Supplying *forecast* temperature, wind, humidity and pressure at the target hour as explicit inputs would replace inference with information, and the SHAP analysis suggests exactly where the gain would land: at +72 h the model already leans on pressure and seasonal phase as proxies for regime, which is precisely the signal a weather forecast would provide directly. The training pipeline would need no structural change – only additional feature columns and a matching change to the serving feature assembly.

Accumulating historical volume. 368 days of history is sufficient to learn a diurnal cycle and one pass through the seasons, but not to distinguish a genuine seasonal pattern from a single-year anomaly. The consequence is visible in the per-fold metrics, where the earliest expanding-window fold trains on one season and is scored on another. This limitation resolves itself: the hourly pipeline adds 24 rows per day unattended, so multi-year coverage accrues without further effort, and each nightly retrain automatically benefits. Once the store spans several years, two things become worth revisiting – predicting the absolute AQI level rather than the delta, and giving sequence models such as the LSTM or a Temporal Fusion Transformer enough data to compete on their own terms.

Averaging windows in the AQI conversion. The EPA defines its breakpoints against averaging windows the data source does not provide – 24 hours for particulates, 8 hours for ozone and carbon monoxide. Instantaneous hourly readings are used as a proxy, which biases the computed AQI high during short spikes. Because concentrations are stored raw and `src/repair_feature_store.py` can recompute the AQI column in place, implementing proper trailing-window averages is a contained change that would improve fidelity to the standard.

Calibrated prediction intervals. The current bands are an honest heuristic combining out-of-sample residual spread with ensemble disagreement, and are labelled as such. Quantile regression forests or conformal prediction would provide intervals with guaranteed empirical coverage, which matters if the forecast is ever used for formal decision-making rather than public information.

14 Conclusion

The Pearls AQI Predictor demonstrates that a complete, continuously-operating machine learning system – ingestion, feature store, automated retraining, model registry, explainability, REST API and interactive dashboard – can be built and run with no managed infrastructure whatsoever. Every scheduled component executes on GitHub Actions, every artefact is versioned in Git, and the user interface redeploys itself from the same repository. The infrastructure cost is zero and there is no server for anyone to administer.

The system’s substance rests on genuine data and honest evaluation. The feature store holds 8,835 consecutive hourly observations covering 368 days, entirely measured, with lineage documented down to SHA-256 checksums and retained raw API responses. Four model families were placed in competition at each of three horizons under expanding-window time-series cross-validation with a horizon-length gap to prevent leakage, and every candidate’s metrics were recorded rather than only the winner’s. Random Forest was selected at all three horizons on out-of-sample RMSE, beating a persistence baseline by 7.0%, 10.5% and 8.1% at +24 h, +48 h and +72 h. Crucially, the architecture would have refused to deploy a model that did *not* beat persistence, falling back to the baseline and saying so.

Beyond the metrics, the design choices that most define the system are the ones that protect correctness: a single shared feature-engineering function used by both training and serving, eliminating skew by construction; a single prediction module behind both the API and the dashboard, making disagreement between them impossible; and SHAP artefacts that are

timestamp-checked against the models they claim to explain. The resulting SHAP analysis shows attribution migrating from recent AQI momentum at +24 h to particulate loading and seasonal regime at +72 h – the physically coherent behaviour a well-specified forecasting model should exhibit, and evidence that the models have learned mechanism rather than coincidence.

Three days of air quality forecasting for a city of twenty million people, on zero infrastructure, retraining itself nightly. The remaining path to improved accuracy is clear and largely mechanical: feed the models forecast weather instead of asking them to infer it, and let the hourly pipeline keep accumulating history.

References

References

- [1] OpenWeather. *Air Pollution API*. <https://openweathermap.org/api/air-pollution>
- [2] OpenWeather. *Air Pollution API – Historical Data*. <https://openweathermap.org/api/air-pollution#history>
- [3] Open-Meteo. *Historical Weather API (ERA5 Archive)*. <https://open-meteo.com/en/docs/historical-weather-api>
- [4] United States Environmental Protection Agency. *Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)*. EPA-454/B-24-002. <https://www.airnow.gov/aqi/aqi-basics/>
- [5] Hopsworks. *Hopsworks Feature Store Documentation*. <https://docs.hopsworks.ai/>
- [6] Hopsworks. *Feature Group Stream Ingestion (stream=True)*. https://docs.hopsworks.ai/latest/user_guides/fs/feature_group/create/
- [7] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/stable/>
- [8] Scikit-learn Developers. *TimeSeriesSplit – Cross-validation for Time Series Data*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html
- [9] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, 265–283. <https://www.tensorflow.org/>
- [10] Chollet, F. et al. (2015). Keras. https://keras.io/api/layers/recurrent_layers/lstm/
- [11] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 4765–4774.
- [12] Lundberg, S. M., Erion, G. G. and Lee, S.-I. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, 2(1), 56–67. <https://shap.readthedocs.io/>
- [13] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [14] Streamlit Inc. *Streamlit Documentation and Streamlit Community Cloud*. <https://docs.streamlit.io/>
- [15] Ramírez, S. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [16] GitHub. *GitHub Actions Documentation – Events that Trigger Workflows (schedule)*. <https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows#schedule>

- [17] Apache Software Foundation. *Apache Parquet Documentation*. <https://parquet.apache.org/docs/>
- [18] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56. <https://pandas.pydata.org/docs/>